

爱吃蟠桃的孙悟空

题目描述

孙悟空爱吃蟠桃，有一天趁着蟠桃园守卫不在来偷吃。已知蟠桃园有 N 棵桃树，每颗树上都有桃子，守卫将在 H 小时后回来。

孙悟空可以决定他吃蟠桃的速度K（个/小时），每个小时选一颗桃树，并从树上吃掉 K 个，如果树上的桃子少于 K 个，则全部吃掉，并且这一小时剩余的时间里不再吃桃。

孙悟空喜欢慢慢吃，但又想在守卫回来前吃完桃子。

请返回孙悟空可以在 H 小时内吃掉所有桃子的最小速度 K（K为整数）。如果以任何速度都吃不完所有桃子，则返回0。

输入描述

第一行输入为 N 个数字，N 表示桃树的数量，这 N 个 **数字表示** 每颗桃树上蟠桃的数量。

第二行输入为一个数字，表示守卫离开的时间 H。

其中数字通过空格分割，N、H为正整数，每颗树上都有蟠桃，且 $0 < N < 10000$ ， $0 < H < 10000$ 。

输出描述

吃掉所有蟠桃的最小速度 K，无解或输入异常时输出 0。

用例

输入	2 3 4 5 4
输出	5
说明	无

输入	2 3 4 5 3
输出	0
说明	无

输入	30 11 23 4 20 6
输出	23
说明	无

题目解析

本题可以使用 **二分法** 解题。

本题和[机器人搬砖](#)

类似。具体解析可以参照上面博客。或者直接看当前博客代码注释，已加入详细注释说明。

JavaScript 算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const cnts = (await readline()).split(" ").map(Number);
7      const h = parseInt(await readline());
8
9      console.log(getResult(cnts, h));
10 })();
11
```

运行

```
12 function getResult(cnts, h) {
13     // 每小时只能选一颗桃树，因此 h 小时最多只能选 h 棵树，如果桃树数量cnts.length > h，那么h小时肯定吃不完
14     if (cnts.length > h) {
15         return 0;
16     }
17
18     // 拥有最多桃子的那颗桃树上的桃子数量
19     let max = Math.max(...cnts);
20
21     // 如果桃树数量就是h棵，那么只能一小时吃完一颗树，才能保证h小时内吃完。此时，吃桃速度至少是max
22     if (cnts.length == h) {
23         return max;
24     }
25
26     // 如果只有1棵桃树，且这颗树上只有1个桃，那么吃桃速度可以是1
27     let min = 1;
28
29     // 当桃树数量少于h棵时，以max速度吃桃肯定可以吃完，但是不一定是最优解
30     let ans = max;
31
32     // 二分法
33     while (min <= max) {
34         // 取中间值作为吃桃速度进行尝试
35         const mid = (min + max) >> 1;
36
37         // 如果以mid速度，可以在h小时内吃完cnts所有桃，那么mid就是一个可能解
38         if (check(mid, h, cnts)) {
39             ans = mid;
40             // 继续尝试更小的速度
41             max = mid - 1;
42         } else {
43             // 以mid速度无法在h小时内吃完cnts所有桃，那么mid就取小了，下次应该取更大的吃桃速度
44             min = mid + 1;
45         }
46     }
47
48     return ans;
```

```

49 | }50 |
51 | function check(speed, limit, cnts) {
52 |     // 已花费时间
53 |     let cost = 0;
54 |
55 |     for (const cnt of cnts) {
56 |         // 以speed速度吃完一颗桃树需要的时间，累加进cost
57 |         cost += Math.ceil(cnt / speed);
58 |
59 |         // 如果已花费时间超过了limit限制，那么说明无法以speed速度在limit时间内吃完所有桃树，此时可以直接返回false
60 |         if (cost > limit) {
61 |             return false;
62 |         }
63 |     }
64 |
65 |     // 可以以speed速度，在limit小时内吃完所有cnts桃树
66 |     return true;
67 | }

```

Java算法源码

```

1 | import java.util.Arrays;
2 | import java.util.Scanner;
3 |
4 | public class Main {
5 |     public static void main(String[] args) {
6 |         Scanner sc = new Scanner(System.in);
7 |
8 |         int[] cnts = Arrays.stream(sc.nextLine().split(" ")).mapToInt(Integer::parseInt).toArray();
9 |         int h = Integer.parseInt(sc.nextLine());
10 |
11 |         System.out.println(getResult(cnts, h));
12 |     }

```

```
13 | 14 | public static int getResult(int[] cnts, int h) {
15 | // 每个小时只能选一颗桃树，因此 h 小时最多只能选 h 棵树，如果桃树数量cnts.Length > h，那么h小时肯定吃不完
16 | if (cnts.length > h) {
17 |     return 0;
18 | }
19 |
20 | // 拥有最多桃子的那颗桃树上的桃子数量
21 | int max = Arrays.stream(cnts).max().orElse(0);
22 |
23 | // 如果桃树数量就是h棵，那么只能一小时吃完一颗树，才能保证h小时内吃完。此时，吃桃速度至少是max
24 | if (cnts.length == h) {
25 |     return max;
26 | }
27 |
28 | // 如果只有1棵桃树，且这颗树上只有1个桃，那么吃桃速度可以是1
29 | int min = 1;
30 |
31 | // 当桃树数量少于h棵时，以max速度吃桃肯定可以吃完，但是不一定是最优解
32 | int ans = max;
33 |
34 | // 二分法
35 | while (min <= max) {
36 |     // 取中间值作为吃桃速度进行尝试
37 |     int mid = (min + max) >> 1;
38 |
39 |     // 如果以mid速度，可以在h小时内吃完cnts所有桃，那么mid就是一个可能解
40 |     if (check(mid, h, cnts)) {
41 |         ans = mid;
42 |         // 继续尝试更小的速度
43 |         max = mid - 1;
44 |     } else {
45 |         // 以mid速度无法在h小时内吃完cnts所有桃，那么mid就取小了，下次应该取更大的吃桃速度
46 |         min = mid + 1;
47 |     }
48 | }
49 |
50 | return ans;
```

```

51 | }
52 |
53 | public static boolean check(int speed, int limit, int[] cnts) {
54 |     // 已花费时间
55 |     int cost = 0;
56 |
57 |     for (int cnt : cnts) {
58 |         // 以speed速度吃完一颗桃树需要的时间，累加进cost
59 |         cost += cnt / speed + (cnt % speed > 0 ? 1 : 0);
60 |
61 |         // 如果已花费时间超过了Limit限制，那么说明无法以speed速度在Limit时间内吃完所有桃树，此时可以直接返回false
62 |         if (cost > limit) {
63 |             return false;
64 |         }
65 |     }
66 |
67 |     // 可以以speed速度，在Limit小时内吃完所有cnts桃树
68 |     return true;
69 | }
70 | }

```

Python算法源码

```

1 | import math
2 |
3 | # 输入获取
4 | cnts = list(map(int, input().split()))
5 | h = int(input())
6 |
7 |
8 | def check(speed):
9 |     # 已花费时间
10 |     cost = 0
11 |
12 |     for cnt in cnts:

```

```

13     # 以speed速度吃完一颗桃树需要的时间，累加进cost
14     cost += math.ceil(cnt / speed)
15
16     # 如果已花费时间超过了Limit限制，那么说明无法以speed速度在Limit时间内吃完所有桃树，此时可以直接返回false
17     if cost > h:
18         return False
19
20     # 可以以speed速度，在Limit小时内吃完所有cnts桃树
21     return True
22
23
24 def getResult():
25     n = len(cnts)
26
27     # 每个小时只能选一颗桃树，因此 h 小时最多只能选 h 棵树，如果桃树数量cnts.Length > h，那么h小时肯定吃不完
28     if n > h:
29         return 0
30
31     # 拥有最多桃子的那颗桃树上的桃子数量
32     maxSpeed = max(cnts)
33
34     # 如果桃树数量就是h棵，那么只能一小时吃完一颗树，才能保证h小时内吃完。此时，吃桃速度至少是max
35     if n == h:
36         return maxSpeed
37
38     # 如果只有1棵桃树，且这颗树上只有1个桃，那么吃桃速度可以是1
39     minSpeed = 1
40
41     # 当桃树数量少于h棵时，以max速度吃桃肯定可以吃完，但是不一定是最优解
42     ans = maxSpeed
43
44     # 二分法
45     while minSpeed <= maxSpeed:
46         # 取中间值作为吃桃速度进行尝试
47         mid = (minSpeed + maxSpeed) >> 1
48
49     # 如果以mid速度，可以在h小时内吃完cnts所有桃，那么mid就是一个可能解

```

```

50         if check(mid):51             ans = mid
51             # 继续尝试更小的速度
52             maxSpeed = mid - 1
53         else:
54             # 以mid速度无法在h小时内吃完cnts所有桃，那么mid就取小了，下次应该取更大的吃桃速度
55             minSpeed = mid + 1
56
57
58     return ans
59
60
61 # 算法调用
62 print(getResult())

```

C算法源码

```

1  #include <stdio.h>
2  #include <limits.h>
3
4  #define MAX(a,b) ((a) > (b) ? (a) : (b))
5
6  #define MAX_SIZE 10000
7
8  int check(int speed, int limit, const int cnts[], int cnts_size) {
9      // 已花费时间
10     int cost = 0;
11
12     for(int i=0; i<cnts_size; i++) {
13         int cnt = cnts[i];
14
15         // 以speed速度吃完一颗桃树需要的时间，累加进cost
16         cost += cnt / speed + (cnt % speed > 0 ? 1 : 0);
17
18         // 如果已花费时间超过了limit限制，那么说明无法以speed速度在limit时间内吃完所有桃树，此时可以直接返回false
19         if(cost > limit) {
20             return 0;

```

运行


```
21     }
22 }
23
24 // 可以以 $speed$ 速度, 在 $Limit$ 小时内吃完所有 $cnts$ 桃树
25 return 1;
26 }
27
28 int getResult(int cnts[], int cnts_size, int h) {
29     // 每个小时只能选一颗桃树, 因此  $h$  小时最多只能选  $h$  棵树, 如果桃树数量 $cnts.Length > h$ , 那么 $h$ 小时肯定吃不完
30     if(cnts_size > h) {
31         return 0;
32     }
33
34     // 拥有最多桃子的那颗桃树上的桃子数量
35     int maxSpeed = INT_MIN;
36     for(int i=0; i<cnts_size; i++) {
37         maxSpeed = MAX(maxSpeed, cnts[i]);
38     }
39
40     // 如果桃树数量就是 $h$ 棵, 那么只能一小时吃完一颗树, 才能保证 $h$ 小时内吃完。此时, 吃桃速度至少是 $max$ 
41     if(cnts_size == h) {
42         return maxSpeed;
43     }
44
45     // 如果只有1棵桃树, 且这颗树上只有1个桃, 那么吃桃速度可以是1
46     int minSpeed = 1;
47
48     // 当桃树数量少于 $h$ 棵时, 以 $max$ 速度吃桃肯定可以吃完, 但是不一定是最优解
49     int ans = maxSpeed;
50
51     // 二分法
52     while(minSpeed <= maxSpeed) {
53         // 取中间值作为吃桃速度进行尝试
54         int mid = (minSpeed + maxSpeed) >> 1;
55
56         // 如果以 $mid$ 速度, 可以在 $h$ 小时内吃完 $cnts$ 所有桃, 那么 $mid$ 就是一个可能解
57         if(check(mid, h, cnts, cnts_size)) {
```

```

58         ans = mid;
59         // 继续尝试更小的速度
60         maxSpeed = mid - 1;
61     } else {
62         // 以mid速度无法在h小时内吃完cnts所有桃，那么mid就取小了，下次应该取更大的吃桃速度
63         minSpeed = mid + 1;
64     }
65 }
66
67 return ans;
68 }
69
70 int main() {
71     int cnts[MAX_SIZE];
72     int cnts_size = 0;
73
74     while(scanf("%d", &cnts[cnts_size++])) {
75         if(getchar() != ' ') break;
76     }
77
78     int h;
79     scanf("%d", &h);
80
81     printf("%d\n", getResult(cnts, cnts_size, h));
82
83     return 0;
84 }

```

C++ 算法源码

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4

```

```

5 | bool check(int speed, int limit, vector<int>& cnts) { 6 | // 已花费时间
7 |     int cost = 0;
8 |
9 |     for (const auto &cnt: cnts) {
10 |         // 以speed速度吃完一颗桃树需要的时间，累加进cost
11 |         cost += cnt / speed + (cnt % speed > 0 ? 1 : 0);
12 |
13 |         // 如果已花费时间超过了limit限制，那么说明无法以speed速度在limit时间内吃完所有桃树，此时可以直接返回false
14 |         if (cost > limit) {
15 |             return false;
16 |         }
17 |     }
18 |
19 |     // 可以以speed速度，在limit小时内吃完所有cnts桃树
20 |     return true;
21 | }
22 |
23 | int solution(vector<int> &cnts, int h) {
24 |     // 每个小时只能选一颗桃树，因此 h 小时最多只能选 h 棵树，如果桃树数量cnts.Length > h，那么h小时肯定吃不完
25 |     if (cnts.size() > h) {
26 |         return 0;
27 |     }
28 |
29 |     // 拥有最多桃子的那颗桃树上的桃子数量
30 |     int max = *max_element(cnts.begin(), cnts.end());
31 |
32 |     // 如果桃树数量就是h棵，那么只能一小时吃完一颗树，才能保证h小时内吃完。此时，吃桃速度至少是max
33 |     if (cnts.size() == h) {
34 |         return max;
35 |     }
36 |
37 |     // 如果只有1棵桃树，且这颗树上只有1个桃，那么吃桃速度可以是1
38 |     int min = 1;
39 |     // 当桃树数量少于h棵时，以max速度吃桃肯定可以吃完，但是不一定是最优解
40 |     int ans = max;
41 |

```

```
42 // 二分法43 | while (min <= max) {
44 // 取中间值作为吃桃速度进行尝试
45 int mid = (min + max) >> 1;
46
47 // 如果以mid速度，可以在h小时内吃完cnts所有桃，那么mid就是一个可能解
48 if (check(mid, h, cnts)) {
49     ans = mid;
50     // 继续尝试更小的速度
51     max = mid - 1;
52 } else {
53     // 以mid速度无法在h小时内吃完cnts所有桃，那么mid就取小了，下次应该取更大的吃桃速度
54     min = mid + 1;
55 }
56 }
57
58 return ans;
59 }
60
61 int main() {
62     vector<int> cnts;
63
64     int count;
65     while (cin >> count) {
66         cnts.emplace_back(count);
67     }
68
69     int h = cnts.back();
70     cnts.pop_back();
71
72     cout << solution(cnts, h) << endl;
73
74     return 0;
75 }
```

